Azure AD Domain Controller Lab

A step by step Azure and Terraform lab, written so a 12 year old can follow along

Windows Server 2022 | Azure | Terraform | Active Directory

This is the merged, corrected final version. It combines the full walkthrough with the fixes found during review, so every command in this document is the safe, working version, not the original with its bugs left in.

What are we even doing here?

Imagine a school. Every student has a name, a locker, and a password to get into the computer lab. Someone has to keep track of all of that. That someone is a computer called a domain controller. It is the boss computer that knows every user, every password, and every rule.

In this lab, we are going to build one of those boss computers ourselves, using Microsoft Azure (a giant computer rental company) and a tool called Terraform (a robot that builds computer stuff for us so we don't have to click a hundred buttons by hand).

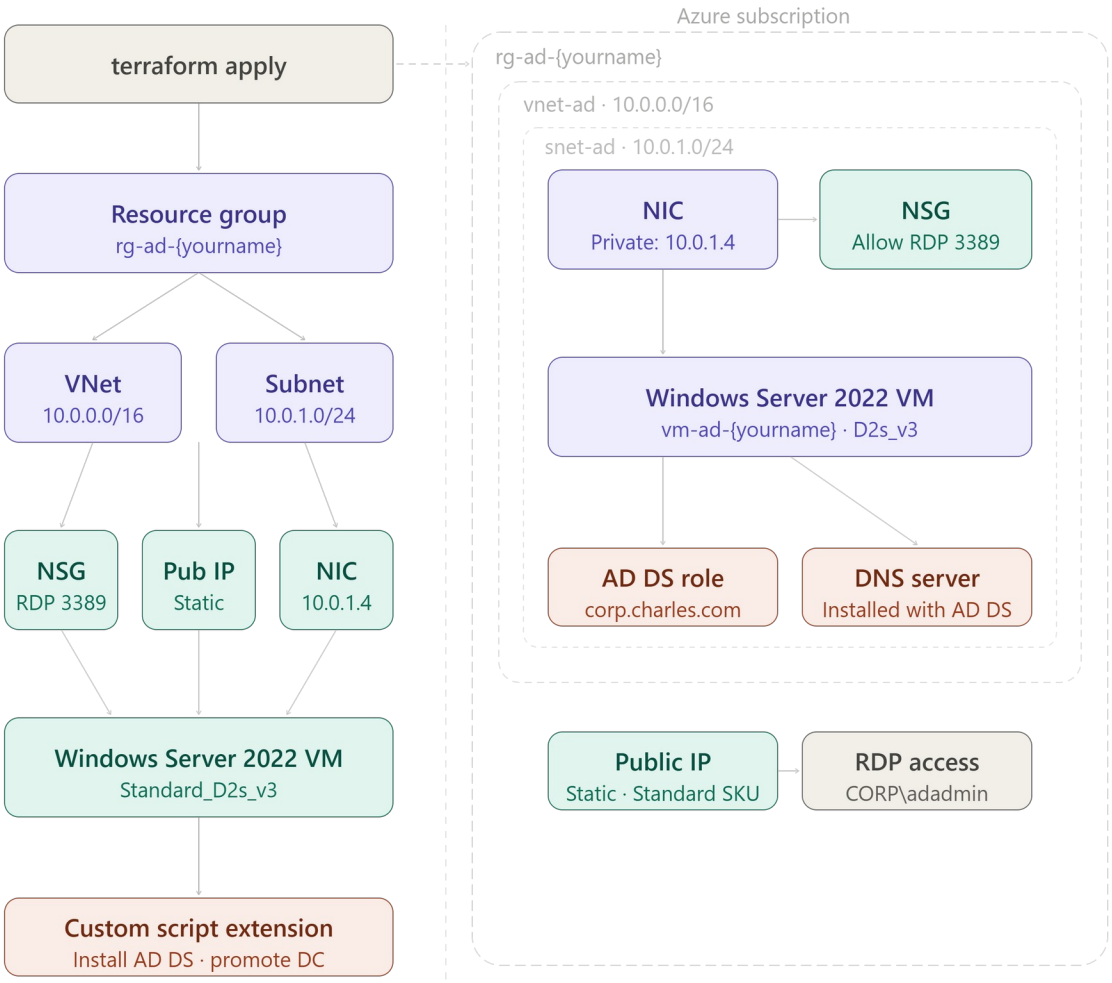
We are not clicking around in a website. We are writing a few text files that describe exactly what we want, and then we tell Terraform to go build it. Think of it like writing down a LEGO instruction booklet, then handing it to a robot that builds the LEGO set for you.

What this actually means:

- Azure is a company that owns giant warehouses full of computers. You can rent one of their computers for a little while.
- Terraform is a tool that reads your instructions and builds things in Azure for you automatically.
- A domain controller is a special server whose job is to know every user account and password on a network, kind of like a hall monitor who has a list of everyone allowed in the building.
- AD DS stands for Active Directory Domain Services. That is the actual program running on the server that does the hall monitor job.

Architecture: What Gets Built, and in What Order

This shows two views side by side: on the left, the order Terraform builds things in, top to bottom, like a recipe. On the right, how those pieces actually nest inside each other once they exist in Azure, like boxes inside boxes inside boxes.



Before You Start: Getting Your Tools Ready

Before we can build anything, we need to install two small programs on your own computer, sign in to Azure, and make one folder to keep our files in. None of this happens inside Azure yet. It all happens on your own laptop or desktop first. Here is the full checklist:

• Install the Azure CLI, a small program that lets you talk to Azure by typing commands instead of clicking around a website.

• Sign in to Azure using that CLI, with a command called az login.

• Install Terraform, version 1.3 or newer, the robot builder tool we talked about.

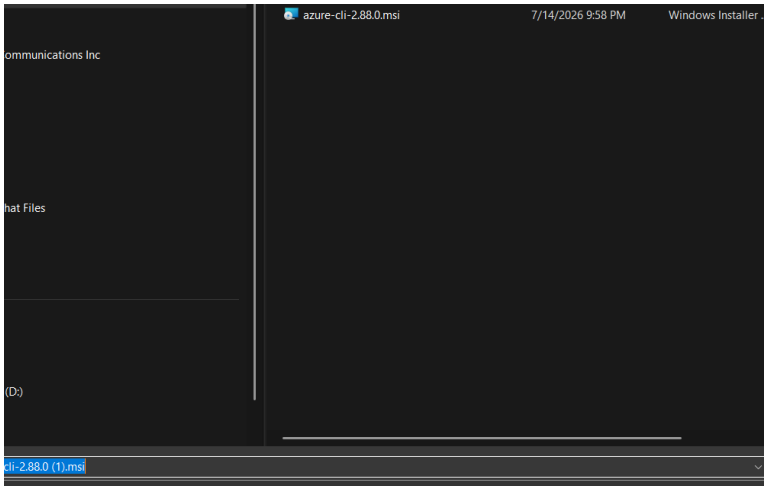
• Create a folder on your computer where we will keep our project files.

Step 1: Install the Azure CLI

The Azure CLI (short for Command Line Interface) is a small program you install once. After that, you can type short commands to ask Azure to do things, instead of clicking through a website. Follow the steps for whichever computer you have.

If you use Windows

1. Open your web browser and go to this address: aka.ms/installazurecliwindows



2. Your browser will download a file that ends in .msi. This is a normal installer file, the same kind of thing you would use to install any other program.

3. Double click the downloaded file to open the installer.

4. Click Next, agree to the license terms, and click Install. Windows may ask for permission to make changes to your computer. Click Yes.

5. When it finishes, click Finish.

6. Open a fresh PowerShell window (search for PowerShell in your Start menu and click it). It has to be a new window, not one you already had open, so it can see the new program.

7. Type az --version and press Enter. If you see version numbers printed on the screen, it worked.

If you use a Mac

1. Open the Terminal app. You can find it by pressing Command + Space, typing Terminal, and pressing Enter.

2. If you do not already have Homebrew installed (a helper tool that installs other programs), paste this in and press Enter:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

3. Once Homebrew is ready, install the Azure CLI by typing:

```
brew update brew install azure-cli
```

4. Check that it installed correctly by typing:

```
az --version
```

If you use Linux

1. Open your terminal application.

2. On Ubuntu or Debian, paste this single line and press Enter. It downloads and runs Microsoft's official install script:

```
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
```

3. Check that it installed correctly by typing:

```
az --version
```

What this actually means:

• The Azure CLI is just a translator. You type a short sentence like az login, and it translates that into the exact request Azure understands, then shows you the answer back in your terminal.

• az --version does not build anything. It just proves the program is installed and tells you which version you have.

• If az is not recognized as a command after installing, close your terminal window completely and open a brand new one. Programs you just installed usually cannot be seen by windows that were already open.

Step 2: Sign in to Azure with az login

Installing the Azure CLI is like installing a key copying machine. Now we actually need to use your key to sign in. Here is exactly what happens, one step at a time:

1. Open your terminal (PowerShell on Windows, Terminal on Mac, or your terminal app on Linux).

2. Type the following and press Enter:

```
az login
```

3. Your default web browser will automatically pop open a Microsoft sign in page. This is the exact same kind of page you would see signing into an Outlook or Xbox account.

4. Type the email address for your Azure account and click Next, then type your password and click Sign in.

5. You might see a message that says Authentication complete. If so, you can close that browser tab and go back to your terminal.

6. Your terminal window will now print a list, showing every Azure subscription your account can use. A subscription is basically your billing account, the thing that gets charged for any computers you rent.

7. If you only have one subscription listed, you are done, it is already selected for you. If you have more than one and need to pick a specific one, type this, using the exact name or ID shown in your list:

```
az account set --subscription "your-subscription-name"
```

8. Double check you are signed in and pointed at the right subscription by typing:

```
az account show
```

What this actually means:

• Signing in this way is exactly like signing into your email on a new device. The browser window is just there to safely handle your password, so your terminal never sees or stores it directly.

• Once you sign in, the CLI remembers you for a while, so you usually will not need to run az login again every single day.

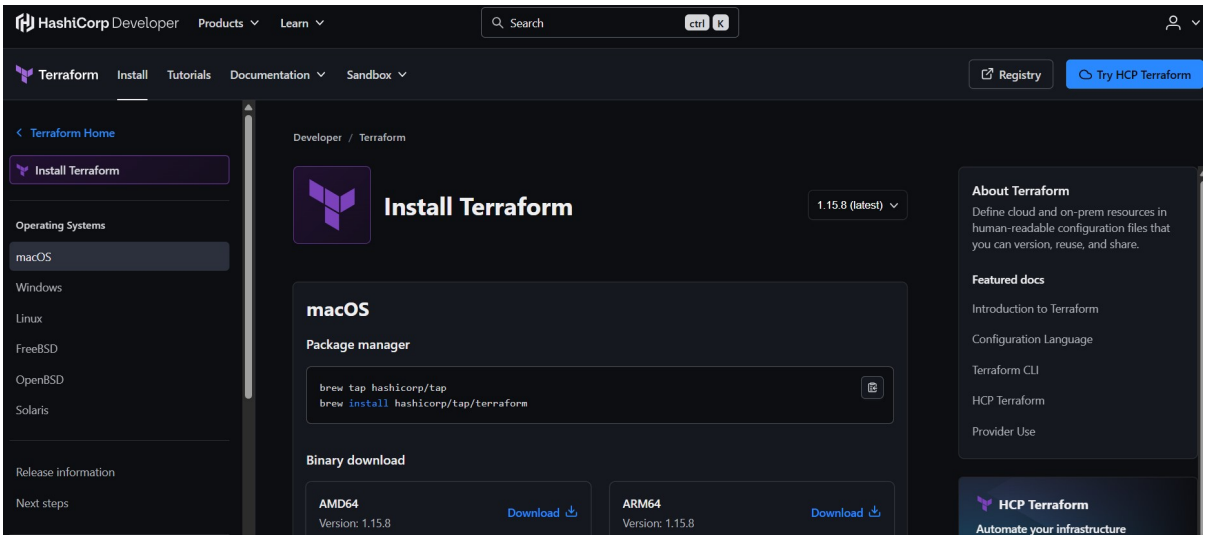
• az account show is just a receipt. It prints back which account and subscription you are currently signed in as, so you can double check before we start spending money building things.

Step 3: Install Terraform, version 1.3 or newer

Terraform is the actual robot builder. It reads the instruction files we write later and builds the real things in Azure. Follow the steps for your computer.

If you use Windows

1. Go to developer.hashicorp.com/terraform/downloads in your browser.

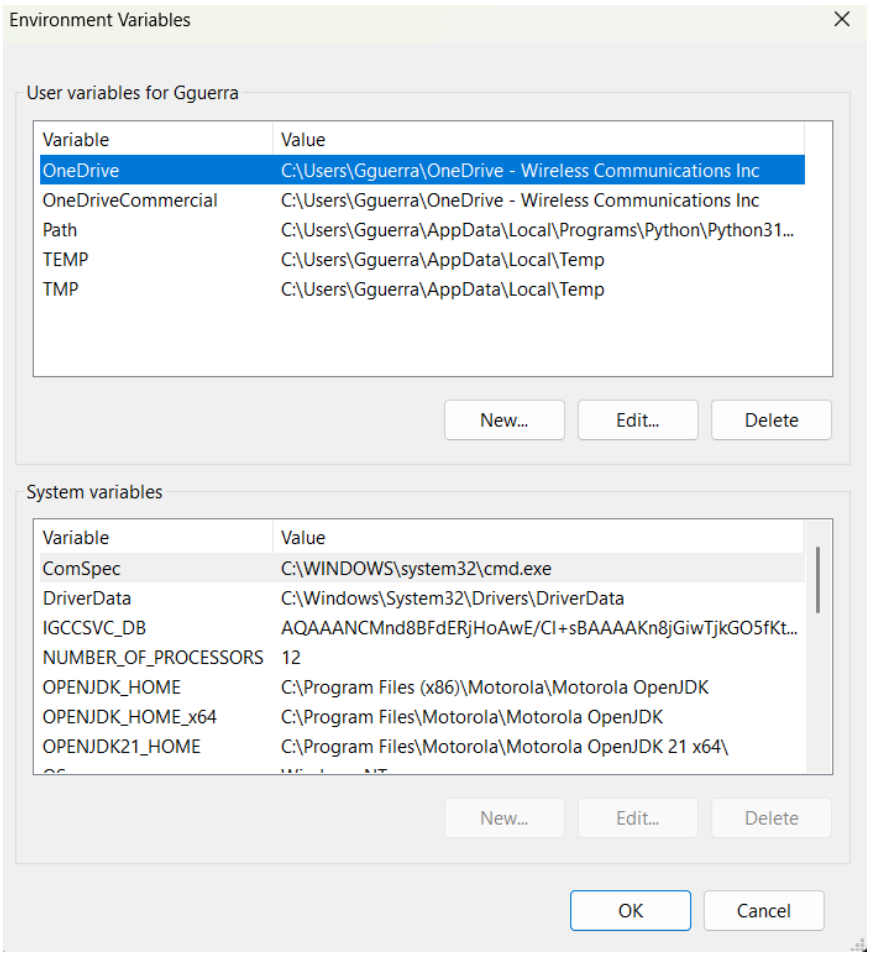


2. Click the download link under the Windows section, choosing the AMD64 option unless you know your computer uses a different chip.

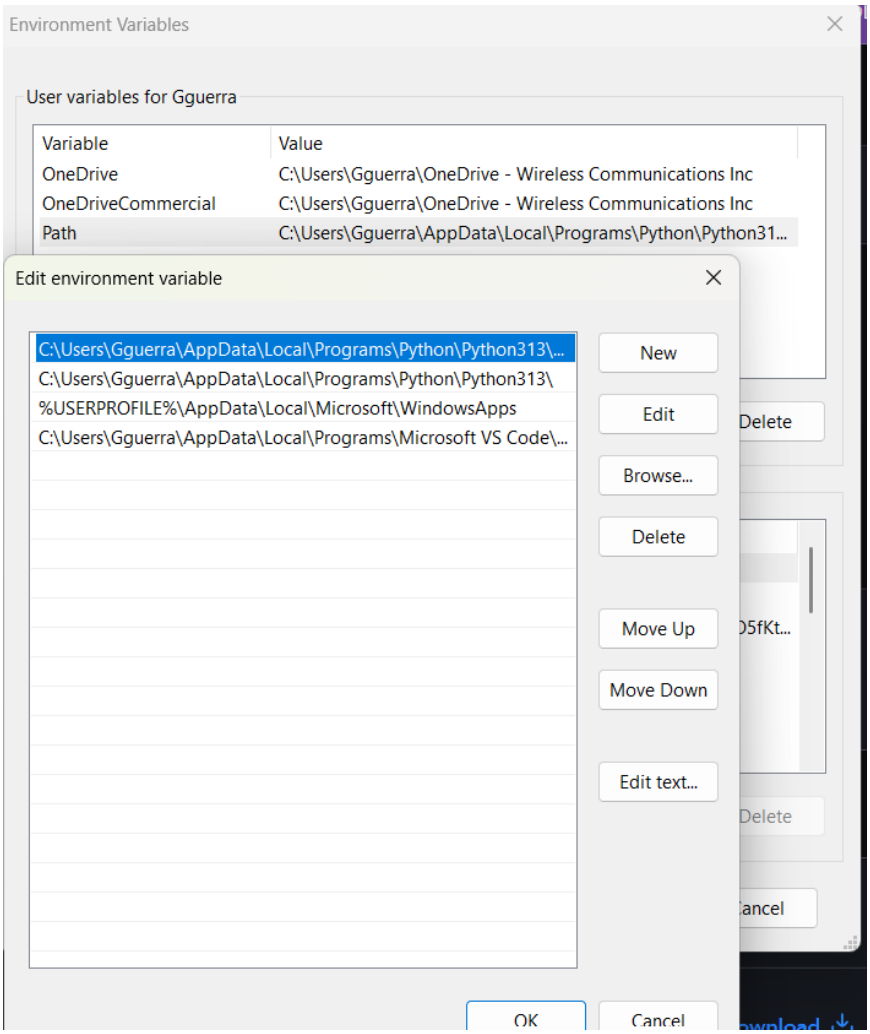
3. This downloads a .zip file. Right click it and choose Extract All, to unzip it. Inside you will find one file named terraform.exe.

4. Create a folder if you do not already have one for programs like this, for example C:\terraform, and move terraform.exe into it.

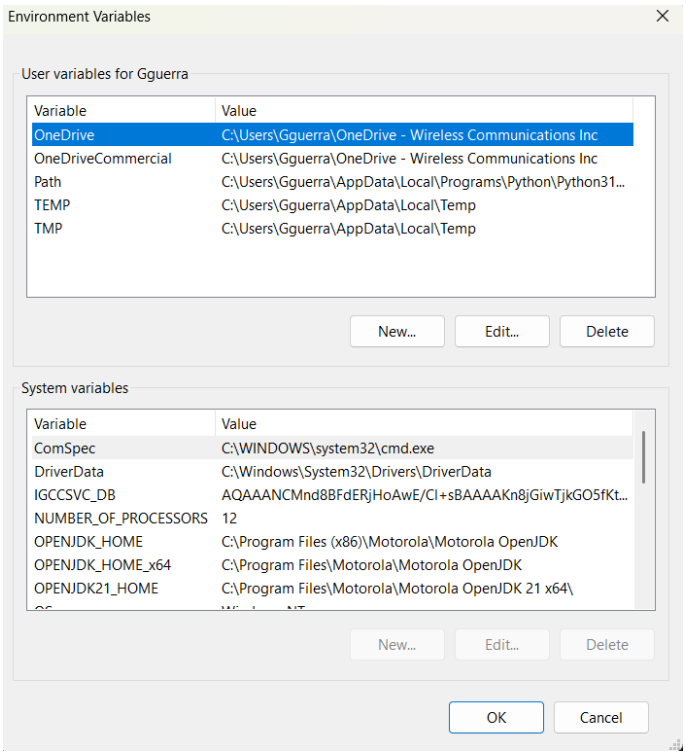
5. Now we need to tell Windows where to find that file. Search for Environment Variables in your Start menu and open Edit the system environment variables.



My own Environment Variables window while adding Terraform to Path.



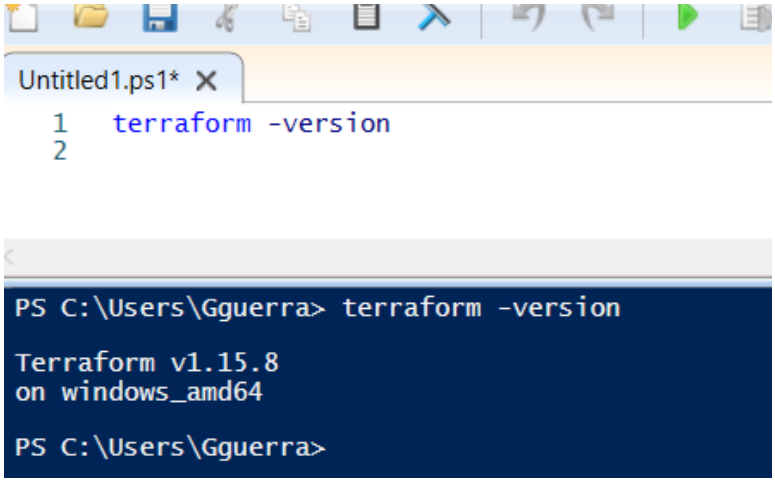
Editing the Path variable to point at C:\\terraform.



6. Click Environment Variables, find the row named Path under User variables, select it, click Edit, click New, and paste in the folder path you used, for example C:\terraform.

7. Click OK on every window to save.

8. Open a brand new PowerShell window and type terraform -version to confirm it works.



My own terraform -version output confirming the install.

If you use a Mac

1. Open Terminal.

2. Type these two lines, one at a time:

```
brew tap hashicorp/tap brew install hashicorp/tap/terraform
```

3. Confirm it installed by typing:

```
terraform -version
```

If you use Linux

1. Open your terminal.

2. On Ubuntu or Debian, paste these lines one at a time. They add HashiCorp's official software source, then install Terraform from it:

```
wget -O- https://apt.releases.hashicorp.com/gpg | gpg --dearmor | sudo tee
/usr/share/keyrings/hashicorp-archive-keyring.gpg echo "deb
[signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/hashicorp.list sudo apt update && sudo apt install
terraform
```

3. Confirm it installed by typing:

```
terraform -version
```

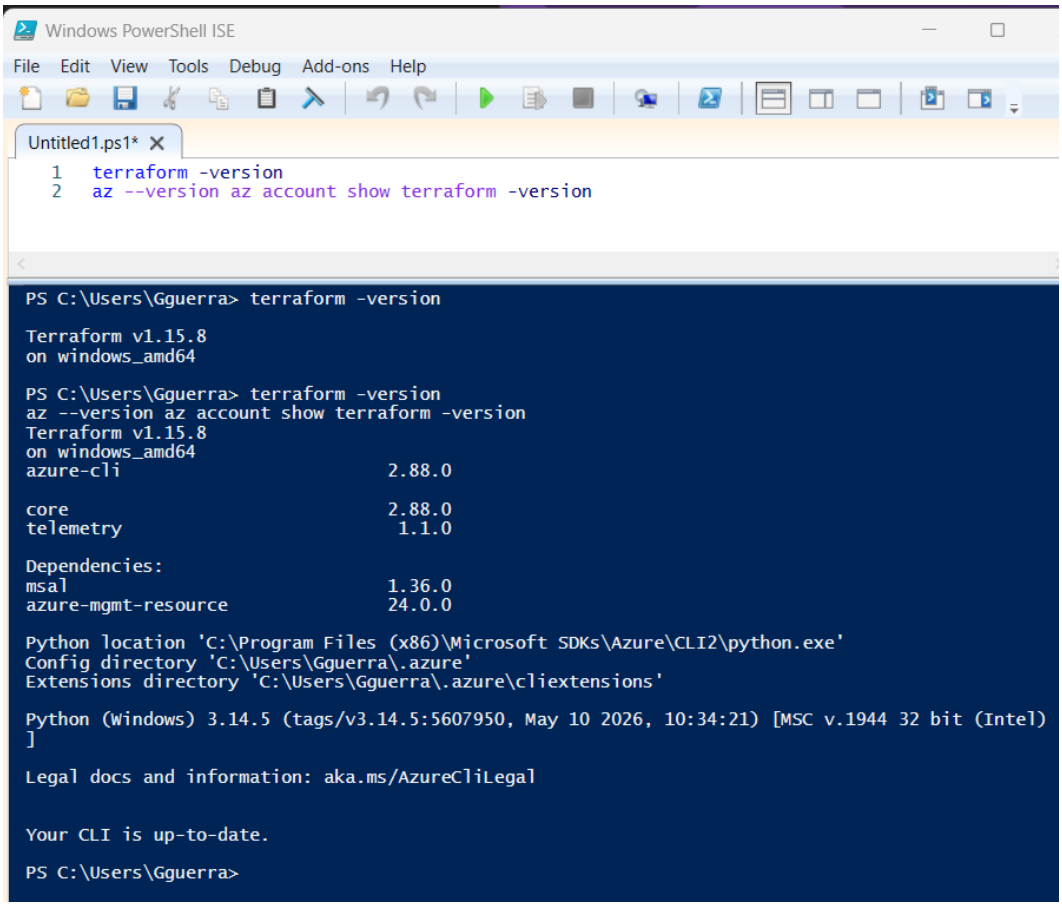
What this actually means:

- The number that prints after terraform -version needs to say 1.3 or higher. If it shows something older, revisit the install steps above and make sure you grabbed the current download.
- Terraform itself does not talk to Azure directly. It uses the sign in information from Step 2 to prove to Azure that it is allowed to build things on your behalf.

Step 4: Make sure everything is actually ready

Before moving on, run through this quick checklist in your terminal. If every command below prints something back instead of an error, your prerequisites are complete:

```
az --version az account show terraform -version
```



My own terminal confirming all three prerequisite checks passed.

If any of those three commands fail:

- az --version failing means the Azure CLI install did not finish. Go back to Step 1.
- az account show failing, or showing no account, means you are not signed in. Go back to Step 2 and run az login again.
- terraform -version failing means Terraform is not installed correctly, or your computer cannot find it. Go back to Step 3 and double check the install location was added correctly.

Part 1: Set Up Your Project Folder

Now that your tools are ready, we need one folder on your computer to keep our Terraform files together. Where you run this next command matters, so pick the option that matches your setup:

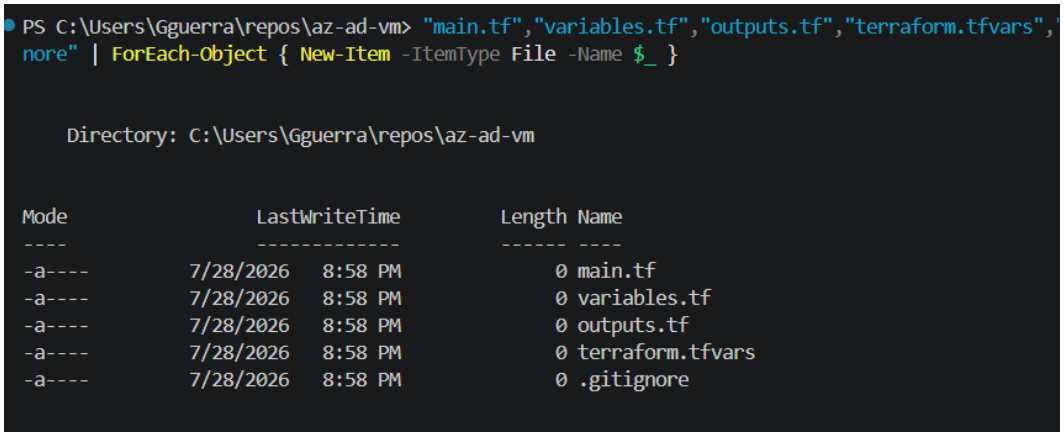
- Mac or Linux: use the Terminal app you already used in Steps 1 through 3.
- Windows using Git Bash, or the Windows Subsystem for Linux (WSL), or Azure Cloud Shell: use that same window.
- Plain Windows PowerShell or Command Prompt: use the separate instructions below, since the command styles are a little different.
- Using VS Code: VS Code has its own built in terminal, and you can type the exact same PowerShell commands there instead of opening a separate window. See the VS Code option below.

On Mac, Linux, Git Bash, WSL, or Azure Cloud Shell, type this single line and press Enter:

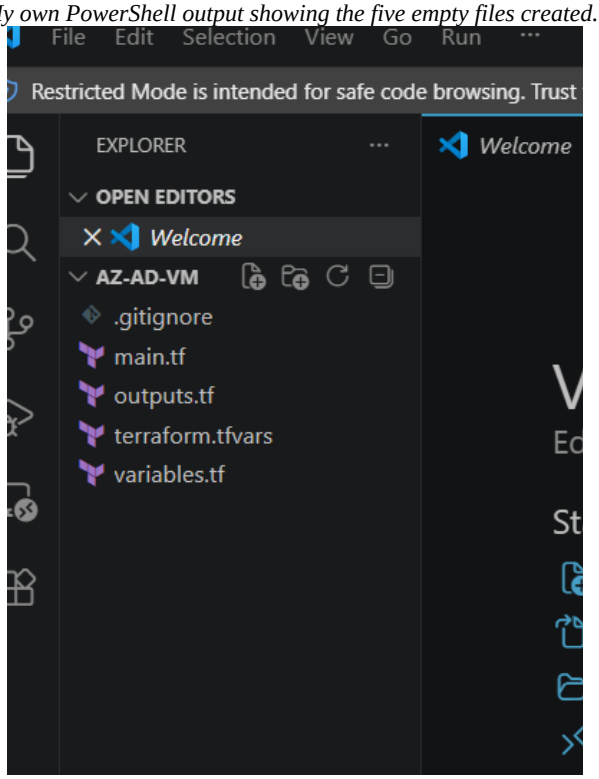
```
mkdir -p ~/repos/az-ad-vm && cd ~/repos/az-ad-vm && touch main.tf variables.tf outputs.tf terraform.tfvars .gitignore
```

On plain Windows PowerShell, type these lines one at a time instead:

```
mkdir C:\repos\az-ad-vm cd C:\repos\az-ad-vm New-Item -ItemType File -Name
main.tf, variables.tf, outputs.tf, terraform.tfvars, .gitignore
```



My own PowerShell output showing the five empty files created.



The same five files shown in VS Code's Explorer sidebar.

FIXED: added .gitignore to the list of files created here. The original draft created it separately later, which made it easy to forget. Since it protects your passwords from ever reaching GitHub, it belongs in this very first batch of files.

If you're using VS Code

1. Open VS Code.

2. Open the built in terminal: click the top menu Terminal, then New Terminal, or press Ctrl+` (the backtick key, usually just above Tab).

3. A terminal panel opens at the bottom. This is the same PowerShell you'd get from the Start menu, just running inside VS Code. Type the same PowerShell commands shown above into it.

4. Once the folder and files exist, go to File, then Open Folder, and select the az-ad-vm folder you just created. All five files will appear in the sidebar on the left, ready to click and edit for Part 2.

A couple of things worth knowing about the VS Code terminal:

• It only picks up a fresh PATH (like the Terraform install from Step 3) if VS Code itself is fully closed and reopened, not just a new terminal tab inside it. If a command like terraform -version says it's not recognized, close VS Code completely and reopen it.

• New-Item with a list of names, like -Name main.tf, variables.tf, outputs.tf, needs the -ItemType File before the -Name for that comma separated list to work correctly. If you type it in the other order, PowerShell may show a ParameterBindingException error.

What this actually means:

• mkdir means make directory. A directory is just another word for folder. The -p just tells it to also make any parent folders that do not already exist, so it will not complain.

• ~/repos/az-ad-vm is the name and location of the new folder. The squiggly line (~) is a shortcut that means your home folder. On plain PowerShell we used C:\repos\az-ad-vm instead, since Windows paths are written with backslashes.

• && means do this next thing after the first thing finishes. It is like saying then in a sentence.

• cd means change directory, which just means step inside a folder.

• touch (or New-Item on PowerShell) creates a brand new empty file. We are creating five empty files: main.tf, variables.tf, outputs.tf, terraform.tfvars, and .gitignore. Right now they are empty. In Part 2 we fill them with instructions.

• Files that end in .tf are Terraform files. Terraform reads every .tf file in the folder as one big set of instructions.

• Everything from here on, every terraform command in this lab, needs to be run while your terminal is sitting inside this same az-ad-vm folder.

Part 2: Filling in the Instruction Files

The five files you just created live inside the az-ad-vm folder on your computer, sitting there completely empty. Now we are going to open each one and paste in some text. This text is written in a language Terraform understands, called HCL (HashiCorp Configuration Language). You do not need to memorize this language. Just paste it in carefully, exactly as shown, and we will explain what each part is doing.

To open and edit these files, use any plain text editor. Notepad on Windows or TextEdit on a Mac (set to plain text mode) will work, though a free code editor like Visual Studio Code is easier to read and is what the rest of this section assumes if you're using it. Do not use Microsoft Word for this, since Word saves hidden formatting that will confuse Terraform.

1. In your file explorer or Finder, browse to the az-ad-vm folder you created in Part 1.

2. Right click main.tf and choose Open with, then pick your text editor.

3. Copy the entire main.tf block shown below and paste it into that empty file.

4. Save the file. Make sure it still ends in .tf and not .txt. On Windows, Notepad sometimes tries to add .txt automatically, so double check the file name afterward.

5. Repeat the same open, paste, and save steps for variables.tf, terraform.tfvars, outputs.tf, and .gitignore, using the matching blocks further down.

If you're using VS Code

1. If you haven't already, go to File, then Open Folder, and select az-ad-vm. All five empty files show up in the Explorer sidebar on the left.

2. Click main.tf once in that sidebar to open it in the editor.

3. Copy the entire main.tf block shown below and paste it in.

4. Save with Ctrl+S (Cmd+S on a Mac). Since the file was already named main.tf when it was created, VS Code will not add a .txt extension the way Notepad sometimes does, so there's nothing extra to check there.

5. Click each of the other four files in the sidebar one at a time and repeat paste and save for variables.tf, terraform.tfvars, outputs.tf, and .gitignore, using the matching blocks further down.

A tip for VS Code specifically:

• If you install the free HashiCorp Terraform extension from the Extensions panel (the icon that looks like four squares on the left sidebar), VS Code will color code the .tf files and catch obvious typos as you paste, which makes it easier to spot a stray bracket or missing quote before you ever run terraform plan.

main.tf, the main blueprint

This is the biggest file. It describes every single piece of computer stuff we want Azure to build for us: a network, a security guard rule, a virtual computer, and the little program that turns that computer into our domain controller. Open the empty main.tf file you created in Part 1 and paste in everything below.

```
terraform {   required_providers {     azurerm = {       source  =
"hashicorp/azurerm"       version = "~> 3.0"     }   } }  provider "azurerm"
{   features {} }  resource "azurerm_resource_group" "main" {   name     = "rg-
ad-${var.yourname}"   location = var.location   tags     = var.tags }  resource
"azurerm_virtual_network" "main" {   name                = "vnet-ad-$
{var.yourname}"   location            = var.location   resource_group_name =
azurerm_resource_group.main.name   address_space       = ["10.0.0.0/16"]   tags
= var.tags }  resource "azurerm_subnet" "main" {   name                 = "snet-
ad"   resource_group_name  = azurerm_resource_group.main.name
virtual_network_name = azurerm_virtual_network.main.name   address_prefixes     =
["10.0.1.0/24"] }  resource "azurerm_public_ip" "main" {   name                =
"pip-ad-${var.yourname}"   location            = var.location
resource_group_name = azurerm_resource_group.main.name   allocation_method   =
"Static"   sku                 = "Standard"   tags                = var.tags }
resource "azurerm_network_security_group" "main" {   name                = "nsg-
ad-${var.yourname}"   location            = var.location   resource_group_name =
azurerm_resource_group.main.name   security_rule {     name
= "allow-rdp"     priority                  = 1000     direction
= "Inbound"     access                    = "Allow"     protocol
= "Tcp"     source_port_range          = "*"     destination_port_range     =
"3389"     source_address_prefix      = var.rdp_source
destination_address_prefix = "*"   }   tags = var.tags }  resource
"azurerm_network_interface" "main" {   name                = "nic-ad-$
{var.yourname}"   location            = var.location   resource_group_name =
azurerm_resource_group.main.name   ip_configuration {     name
= "internal"     subnet_id                     = azurerm_subnet.main.id
private_ip_address_allocation = "Static"     private_ip_address            =
"10.0.1.4"     public_ip_address_id          = azurerm_public_ip.main.id   }
tags = var.tags }  resource
"azurerm_network_interface_security_group_association" "main"
{   network_interface_id     = azurerm_network_interface.main.id
network_security_group_id = azurerm_network_security_group.main.id }  resource
"azurerm_windows_virtual_machine" "main" {   name                  = "vm-ad-$
{var.yourname}"   computer_name         = "ad-${var.yourname}"   location
= var.location   resource_group_name   = azurerm_resource_group.main.name   size
= "Standard_D2s_v3"   admin_username        = var.admin_username   admin_password
= var.admin_password   network_interface_ids =
[azurerm_network_interface.main.id]   os_disk {     caching              =
"ReadWrite"     storage_account_type = "Premium_LRS"     disk_size_gb         =
127   }   source_image_reference {     publisher = "MicrosoftWindowsServer"
offer     = "WindowsServer"     sku       = "2022-Datacenter"     version   =
```

```
"latest"   }   tags = var.tags }  resource "azurerm_virtual_machine_extension"
"ad_setup" {   name                 = "install-ad-ds"   virtual_machine_id   =
azurerm_windows_virtual_machine.main.id   publisher            =
"Microsoft.Compute"   type                 = "CustomScriptExtension"
type_handler_version = "1.10"   protected_settings =
jsonencode({     commandToExecute = "powershell -ExecutionPolicy Unrestricted -
Command \"Install-WindowsFeature -Name AD-Domain-Services -
IncludeManagementTools; Import-Module ADDSDeployment; Install-ADDSForest -
DomainName '${var.domain_name}' -DomainNetbiosName '${var.domain_netbios}' -
ForestMode 'WinThreshold' -DomainMode 'WinThreshold' -InstallDns:$true -
SafeModeAdministratorPassword (ConvertTo-SecureString '${var.dsrm_password}' -
AsPlainText -Force) -Force:$true\""   })   tags = var.tags }
```

FIXED: three changes from the original draft. 1) source_address_prefix now uses var.rdp_source instead of a wildcard "*", so the front door isn't left open to the whole internet. 2) The startup script's settings moved from settings to protected_settings, so the dsrm_password inside it is encrypted instead of sitting in plain text where anyone checking on the VM's status could read it. 3) The additional_unattend_content (AutoLogon) block from the original draft has been removed entirely, since it served no real purpose here and left a copy of the admin password in a plain text file on the VM's own disk.

What this actually means:

- The terraform block tells Terraform to use Azure building blocks from HashiCorp, version 3.
- azurerm_resource_group creates an Azure folder to organize everything and delete it easily later.
- azurerm_virtual_network and azurerm_subnet create the private network for the server.
- azurerm_public_ip gives the server an internet address so you can reach it.
- azurerm_network_security_group is a security guard. The rule inside it, called allow-rdp, tells the guard to let in a specific kind of visit called RDP on port 3389 (think of a port as a specific numbered door), but only from the address you set in rdp_source, not from anyone.
- azurerm_network_interface is like a network cable plugged into the back of the virtual computer, connecting it to the private road system.
- azurerm_windows_virtual_machine is the actual computer we are renting. We are asking for a computer running Windows Server 2022, with a username and password we set ourselves in a different file.
- azurerm_virtual_machine_extension is a little helper script that runs automatically right after the computer turns on for the first time. Ours tells the new computer: install the Active Directory program, then turn yourself into a brand new domain controller using the domain name and passwords we picked. Using protected_settings instead of settings keeps that password-containing instruction encrypted.

variables.tf, the fill in the blanks list

Instead of typing the same words over and over in main.tf, we use placeholders called variables. This file lists every placeholder and describes what it is for. It also lives in the same az-ad-vm folder. Open the empty variables.tf file and paste in everything below.

```
variable "yourname" {   description = "Your name, used to make resource names
unique."   type        = string }  variable "location" {   description = "Azure
region."   type        = string   default     = "eastus" }  variable
"admin_username" {   description = "Local admin username for the VM."   type
= string   default     = "adadmin" }  variable "admin_password" {   description =
"Local admin password for the VM."   type        = string   sensitive   = true }
variable "dsrm_password" {   description = "Directory Services Restore Mode
password for AD DS."   type        = string   sensitive   = true }  variable
"domain_name" {   description = "Fully qualified domain name (e.g.
corp.example.com)."   type        = string   default     = "corp.example.com" }
variable "domain_netbios" {   description = "NetBIOS name for the domain (max 15
characters)."   type        = string   default     = "CORP" }  variable
"rdp_source" {   description = "Your own public IP address in CIDR format, e.g.
```

```
1.2.3.4/32. Find it at whatismyip.com. Never leave this as a wildcard."   type
= string }  variable "tags" {   description = "Tags to apply to all resources."
type        = map(string)   default = {     project = "ad-lab"   } }
```

FIXED: added two variables that weren't in the original draft: admin_username (so the username lives in one place instead of being retyped) and rdp_source (so the front door rule from main.tf has something safe to point at).

What this actually means:

- Think of this file as a form with blank lines on it, like Name: _____ and Password: _____. main.tf just says fill in var.yourname wherever you see it, and this file explains what var.yourname actually means.
- sensitive = true just means Terraform will hide that value when it prints things out on screen, kind of like how a password box on a website shows dots instead of letters. It does not encrypt the value everywhere it's used, which is why we also moved the extension's settings to protected_settings in main.tf.
- Some variables have a default value already picked out, like location defaulting to eastus, which is an Azure data center location in Virginia, USA. You can leave those as is or change them.
- rdp_source has no default on purpose. Terraform will refuse to build anything until you provide your own value for it in terraform.tfvars, which stops you from accidentally forgetting to set it.

terraform.tfvars, your personal answers

This is where you actually fill in the blanks from the form above with your own real answers, like your name and your chosen passwords. This file also lives in the az-ad-vm folder. Open the empty terraform.tfvars file and paste this in, then change the values below to your own before you save:

```
yourname       = "charles" location       = "eastus" admin_password =
"YourPassword123!" dsrm_password  = "YourDSRMPassword123!" domain_name    =
"corp.charles.com" domain_netbios = "CORP" rdp_source     = "YOUR_PUBLIC_IP/32"
```

FIXED: added the rdp_source line. Go to whatismyip.com in a browser, copy the number it shows you, and add /32 to the end, for example 73.14.22.101/32. This is what actually closes the open door from Issue 2. Never leave this as a wildcard.

Everything shown here is a placeholder built from the example name charles.

- Swap every value out for your own name, your own passwords, and your own domain, and keep it consistent everywhere it appears later in the lab (the RDP login, the troubleshooting commands, and so on).

Important note about the DSRM password:

- The dsrm_password is not the same as your admin_password. It is a special emergency password used only if the domain controller ever needs deep repairs.
- Write it down somewhere safe. Once the server is built, there is no button to look it back up.

outputs.tf, the answers Terraform will hand back to us

After Terraform finishes building everything, we want it to tell us a few useful facts, like the new computer's internet address. This file also lives in the az-ad-vm folder. Open the empty outputs.tf file and paste in everything below:

```
output "public_ip" {   description = "Public IP, use this to RDP into the domain
controller"   value       = azurerm_public_ip.main.ip_address }  output
"domain_name" {   description = "Active Directory domain name"   value       =
var.domain_name }  output "admin_username" {   description = "Local admin
username"   value       = var.admin_username }
```

FIXED: admin_username now prints from var.admin_username instead of a hardcoded "adadmin" string, so it automatically stays correct even if you ever change the username in terraform.tfvars.

What this actually means:

• An output is just Terraform's way of saying, here is a piece of information you might want to write down or copy, after I finish building everything.

.gitignore, keeping your passwords off GitHub

If you ever put this project folder under version control with git and push it to GitHub, this file stops your passwords from going with it. Open the empty .gitignore file and paste in everything below:

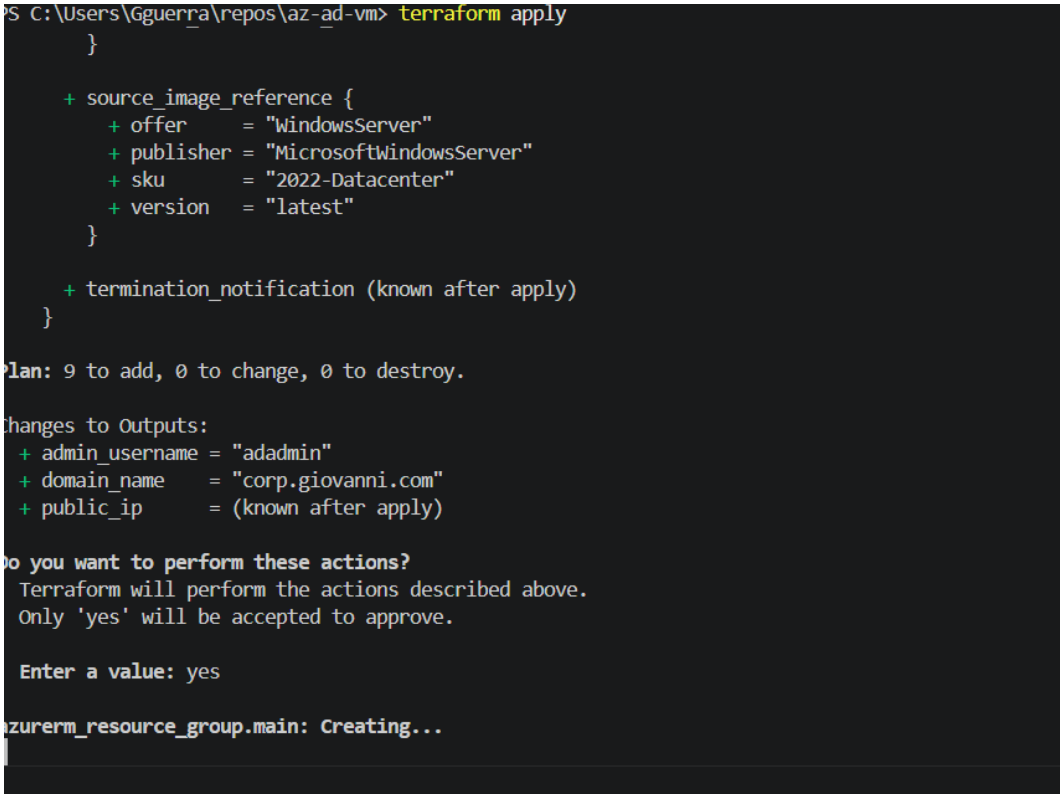
```
terraform.tfvars .terraform/ *.tfstate *.tfstate.backup
```

Do this before your very first git push, not after. This is the single most common beginner mistake with Terraform: committing terraform.tfvars with real passwords in it to a public repository.

Part 3: Actually Building It

Now that all five files are filled in and saved, go back to your terminal. Make sure you are still sitting inside the az-ad-vm folder from Part 1. Then run these three commands one at a time:

```
terraform init terraform plan terraform apply
```



My own terraform apply run: plan summary, yes confirmation, and the first resource being created.

What this actually means:

• terraform init is like unpacking your tools before you start a project. It downloads the pieces Terraform needs to talk to Azure. You only need to run this once per folder, or any time you change the required_providers block.

• terraform plan is Terraform's way of showing you a preview. It reads through your files and tells you exactly what it is about to build, without actually building anything yet. It is like reading the last page of the instruction booklet before you start snapping LEGO pieces together, just to be sure. You should see around 9 resources listed.

• terraform apply is the real thing. It asks you to confirm by typing yes, and then it actually goes and builds everything in Azure.

This step takes some patience. The computer itself takes about 5 to 8 minutes to build. After that, the little helper script takes another 3 to 5 minutes to install the domain controller software, and then the computer restarts on its own. Expect roughly 10 minutes in total before everything is fully ready.

Part 4: Connecting to Your New Server

Once terraform apply is completely finished, ask Terraform for the internet address it gave your new computer:

```
terraform output public_ip
```

Now open your Remote Desktop app, which lets you see and control another computer's screen from far away, and type in that address. You will be asked for a username and password. This part can trip people up, so pay attention to which username style you use:

Method	Username	When to use it
Domain prefix	CORP\adadmin	Try this one first
UPN format	adadmin@corp.charles.com	If the first one does not work
Local account	.\adadmin	Only if the domain setup failed

What this actually means:

• Before the server became a domain controller, it just had a normal local account, and you could log in with a plain username.

• Once it becomes a domain controller, it now thinks of the username as belonging to a whole domain (a group of computers), so you have to say which domain it belongs to. That is why you now type CORP\adadmin, meaning the user named adadmin who belongs to the CORP domain.

Do not rush the reboot:

• After AD DS installs, the server restarts by itself. Wait 5 to 10 minutes after terraform apply finishes before you try to connect.

• If you connect too early, you might just see a black screen or the connection might fail. That is normal. Just wait a bit and try again.

If the connection is refused outright:

• Double check the rdp_source value you set in terraform.tfvars matches the IP address you're actually connecting from right now. If your internet address changed since you built the lab (common on home Wi-Fi), update rdp_source and run terraform apply again to let the new address in.

Part 5: Checking That Everything Worked

Once you are connected and looking at the server's screen, open PowerShell as an Administrator (PowerShell is just a black window where you type commands, similar to the terminal on your own computer, but built into Windows). Then run these four checks, one at a time:

```
Get-Service NTDS | Select-Object Name, Status Get-ADDomain Get-ADDomainController
-Filter * Resolve-DnsName corp.charles.com
```

FIXED: the first line's pipe character has been corrected. The original draft had a lookalike vertical bar character in that spot instead of a real keyboard pipe (|), which would have made the command fail exactly as written.

What this actually means:

• Get-Service NTDS checks whether the actual domain controller program (called NTDS) is turned on and running. Status should say Running.

• Get-ADDomain asks the server to describe the domain it is now in charge of, so you can double check the name and settings look right.

• Get-ADDomainController -Filter * lists every domain controller on the network. Right now that should just be your one new server.

• Resolve-DnsName corp.charles.com checks that the server's built-in phone book (DNS) can correctly look up the domain name and turn it into an address.

If all four commands run without any red error text, congratulations, your domain controller is alive and working.

If Something Goes Wrong

From your own computer, not the server, you can check on the helper script's progress with this command:

```
az vm extension show --resource-group rg-ad-charles --vm-name vm-ad-charles --
name install-ad-ds --query "provisioningState" --output tsv
```

Problem	Why it happens	How to fix it
My old password stopped working	The server restarted and switched over to domain login	Use CORP\adadmin or adadmin@corp.charles.com instead
Extension status says Failed	The setup script hit an error partway through	Look at C:\WindowsAzure\Logs on the server, then run terraform apply again
Black screen when connecting	The server is still restarting	Wait about 5 more minutes and try connecting again
Get-ADDomain says it cannot be found	The Active Directory toolkit is not loaded yet in this window	Run Import-Module ActiveDirectory, then try again
Connection refused entirely	Your current IP doesn't match rdp_source	Update rdp_source in terraform.tfvars to your current IP and run terraform apply again

Field Notes: Real Issues From My Own Build

The steps above are the clean, corrected path. Here is what actually happened during my own build of this lab, and how each issue got resolved, in the order I hit them.

Problem	Cause	Fix
main.tf was empty when Step 3 began	The config text had been pasted into the editor tab but never saved with Ctrl+S before the file was checked	Reopen main.tf, paste the configuration back in, save immediately with Ctrl+S, and confirm the line count with Get-Content main.tf | Measure-Object -Line
terraform plan returned instantly with no output at all	The command was run in Windows PowerShell ISE instead of VS Code's integrated terminal. ISE does not reliably display Terraform's console output	Run all Terraform commands from VS Code's own terminal, opened with Ctrl+backtick inside VS Code, not a separate ISE window
terraform validate failed with Missing required provider	terraform init had not been run yet in this folder, so the azurerm provider plugin was never downloaded	Run terraform init once, then run terraform validate again
terraform plan hung for over 30 seconds with no output	The azurerm provider was waiting on Azure's automatic resource provider registration check, which stalled on Microsoft.DataMigration	Add skip_provider_registration = true inside the provider azurerm block in main.tf. This is the version 3 provider syntax. Version 4 of the provider uses resource_provider_registrations = none instead
Error acquiring the state lock	An earlier interrupted Terraform run left two orphaned terraform-provider-	Find the orphaned processes with Get-Process, filtering for terraform

	azurerm processes running in the background, which kept the state file locked	in the name, then stop them with Stop-Process -Force. Stopping the process released the lock automatically, so the lock file did not need to be deleted by hand
The plan showed source_address_prefix = YOUR_PUBLIC_IP/32	The rdp_source placeholder in terraform.tfvars was never replaced with a real IP address	Look up the real public IP address and update rdp_source in terraform.tfvars before applying
The four Part 5 verification commands were pasted in as one single line	PowerShell read the whole pasted block as one command, so it tried to hand Get-ADDomain to Select-Object as an argument instead of running it separately	Run each of the four commands on its own line, one at a time, instead of pasting them together. All four passed once run individually

What this actually means:

• A file that looks saved in an editor tab is not the same as a file saved to disk. Always confirm with a line or word count before trusting that a paste actually landed.

• PowerShell ISE and VS Code's terminal are two different programs. Even though both run PowerShell, ISE can swallow the colored output that Terraform depends on to show results, which looks exactly like a frozen command.

• A stuck terraform plan is often Azure, not Terraform. Provider registration checks and orphaned background processes both cause the exact same symptom, a command that never seems to finish.

• None of these were show stopping problems. Every one of them had a plain command line fix, and the lab still finished with a working domain controller at the end.

Cleaning Up When You Are Done

Renting computers from Azure is not free, so when you are finished playing around with your domain controller, you should shut it down for good. One command tears down everything we built, run from inside the az-ad-vm folder:

```
terraform destroy
```

Why this step matters:

• This deletes the resource group and everything inside it, meaning the virtual computer, its hard drive, the network card, the public address, the security guard rule, and the private road system.

• If you skip this step, Azure keeps charging you for the rented computer even while you are not using it. Always run terraform destroy when a lab is finished.

• Expect this to take a few minutes, not seconds, since deleting a virtual computer and its disk isn't instant.

And that's it. You just built, connected to, checked on, and tore down your very own domain controller, using nothing but a few text files and a couple of commands.

Bonus: A Few More Words Worth Knowing

These didn't come up directly in the walkthrough above, but you'll run into them as you go further with this.

Word	What it means, in plain words
Bastion	A secure doorway Azure offers for remotely controlling a computer without ever giving that computer a public address at all, unlike this lab's approach of opening port 3389 directly.
Key Vault	A digital safe Azure provides for storing passwords and secrets, so they never sit in a plain text file like terraform.tfvars does in this lab.
Remote state	Instead of Terraform's "what I've already built" notes living only on your own laptop, they're saved in shared cloud storage so any teammate's computer can see the same notes.
Idempotency	A fancy word meaning: running the same instructions twice in a row gives you the same safe result both times, instead of building duplicates or breaking things.
protected_settings vs settings	Two places to put configuration for a startup script extension. settings is stored in the open. protected_settings is encrypted. Anything with a password belongs in protected_settings, which is exactly the fix applied to main.tf above.

Bonus: Key Terraform Ideas Worth Understanding

These are small details in the code that job interviewers tend to ask about, because they show you understand why the code is written the way it is, not just that you copied it.

Idea	Why it's there, in plain words
jsonencode({...})	Turns a normal-looking settings list into the exact text format (JSON) that Azure requires for the startup script's instructions.
sensitive = true	Tells Terraform not to print this value on the screen or in logs. It's a privacy screen, not a lock. It doesn't stop the value from ending up somewhere else if you put it in the wrong spot, which is why protected_settings still matters separately.
Resource references create automatic ordering	When one resource's settings mention another resource's ID, like the NIC referencing the subnet's ID, Terraform automatically knows to build the subnet first. Nobody had to write that order down by hand.
tags = map(string)	A label maker for resources, letting you stick the same set of labels on everything at once instead of typing them repeatedly.

Bonus: How This Lab Would Change for a Real Company

This table shows the shortcut this lab takes for learning purposes, and what a real company would do instead.

Lab shortcut	Production grade version
Passwords typed into terraform.tfvars	Passwords stored in Key Vault, pulled in automatically instead of typed anywhere
Terraform's notes saved only on your own laptop	Notes saved in shared cloud storage (remote state) so a whole team can work from the same source of truth
Front door (RDP) open directly to your one IP	A Bastion doorway used instead, so the computer never has a public address to attack in the first place
Only one front desk computer (one Domain Controller)	At least two, in different locations, so if one goes down the other keeps things running
You personally type terraform apply	A pipeline (an automatic assembly line, like GitHub Actions) runs it for you, with reviews and checks built in